# AI Agent Lab — Project Draft and Journey Story (EN)

> *Historical project perspective: source paths reflect the reorganized layout; proposed work is not an approval or a current status claim. See* `docs-md/continuation.md`.

## Executive Summary

AI Agent Lab is not intended to be only a code-generating chat bot. It is intended to be a platform that transforms natural-language product requests into traceable software delivery workflows.

This article presents the project as a product story:

- Where we started (local prototype)
- Where we are going (multi-agent orchestration platform)
- How we will get there (MVP-first phases)

Core approach: **stabilize a working core first, then scale capabilities in controlled phases**.

---

## Story: “From one task flow to a platform”

We already have a useful execution core:

- A task is created,
- approved,
- processed by a worker,
- and the result is persisted.

This is not yet a full platform, but it is the right foundation. The next step is to build product capabilities around this core:

- orchestration,
- multi-agent delegation,
- human approval,
- quality gates,
- observability and cost control.

---

## Phases (proposed narrative)

## Phase 1 — Hardening the working core

Goal:

- Extend task model,
- formalize transition rules,
- establish testing baseline.

Outcome:

- More reliable and measurable local execution core.

## Phase 2 — Orchestration layer

Goal:

- Orchestrator to convert user requests into plans,
- dependency-aware task execution and approval flow.

Outcome:

- A visible chain of “Request -> Plan -> Approval -> Execution”.

## Phase 3 — Agent workforce

Goal:

- Separate Agent Definition vs Agent Instance,
- assign work to specialized roles.

Outcome:

- Transition from single-worker behavior to multi-agent collaboration.

## Phase 4 — Product interface and governance

Goal:

- Chat + dashboard experience,
- audit trail and decision records,
- quality and security gates.

Outcome:

- Product-owner-centric and governable platform experience.

## Phase 5 — Cloud transition readiness

Goal:

- Prepare transition from local-first prototype to cloud operations,
- keep provider independence as a first-class principle.

Outcome:

- Architecture that can evolve across AWS/Azure and similar providers when needed.

---

# Technical Appendix

## As-is technical anchor points

- Control Plane API: `src/ControlPlane/Program.cs`
- Worker runtime: `src/AgentWorker/Program.cs`
- Local orchestration: `docker-compose.yml`
- Lifecycle scripts: `scripts/local.ps1`, `scripts/local.sh`
- Onboarding and architecture docs: `docs/*`

## Principle highlights

- Human approval gates are mandatory.
- Untrusted agent output must be validated before acceptance.
- Provider lock-in must be minimized.
- Local prototype behavior does not define cloud production architecture by itself.

---

## Conclusion

This project targets a larger outcome than a simple coding assistant: a governable platform that can plan, execute, and validate software delivery workflows.

In the next separate document, we will cover:

- local development operating model,
- phase-2 cloud transition,
- hosting and monthly maintenance costs,
- and provider price/performance re-evaluation criteria.